

OMI Canon Runtime Atlas

Envelope, Orbit, Geometry, Sense, Transport, and Receipt Boundaries

Status: Canon-facing draft

Intended repo: `omi-canon`

Source basis: current OMI C runtime slice: `omienv`, `sector`, `stream`, `omi_orbit`, `omi_pg`, `omi_sense`, `omicron`, `omi_transport`, `omi_receipt`, `omi_omion`.

0. Purpose

This document records the canonical runtime shape implied by the current OMI implementation files.

It does not claim that the runtime is the whole canon.

The correct repository split is:

```
omi-canon
  names invariants and canonical vocabulary

omi-axioms
  carries proof lineage and formal statements

omi-isa
  implements bounded runtime mechanics

omi-lisp
  declares candidate shapes

omi-port
  derives dormant source-target route candidates

transport/adapters
  move envelopes only
```

The runtime files show a coherent architecture:

```
512-bit envelope
→ 512-byte sector
→ stream parser
```

- bitboard witness
- orbit / projective / sense transforms
- transport carrier
- receipt projection

The central invariant remains:

Control is not acceptance.
Scope is not acceptance.
Projection is not acceptance.
Transport is not acceptance.
Validation and receipt are separate authority layers.

1. Canonical Envelope Layer

The canonical runtime cell is the 512-bit OMI envelope:

```
typedef struct {  
    uint8_t gauge[8];  
    uint8_t orientation[8];  
    uint8_t recovery[8];  
    uint8_t target[8];  
    uint8_t relation[32];  
} OMI_512_Envelope;
```

The implementation defines:

OMI_ENV_SIZE = 64 bytes
OMI_PREHEADER_SIZE = 8 bytes
relation[32] = 32-byte relation surface

The canonical preheader is:

FF 00 1C 1D 1E 1F 20 FF

The envelope validator checks:

gauge equals canonical preheader
gauge[0] == gauge[7]

```
gauge[0] >= 0xF0
gauge[2..5] == 1C 1D 1E 1F
gauge[6] == 0x20
```

This maps directly to the pre-language control ladder:

```
00 = NUL
1C = FS
1D = GS
1E = RS
1F = US
20 = SP
```

The envelope therefore stages OMI memory topology before ordinary interpretation.

2. Bitboard Witness Layer

The envelope can be reduced into a 64-bit bitboard witness.

The bitboard tracks:

```
gauge code
D+ diagonal count
D- diagonal count
1E closure bridge
78 system witness
7F seal
7C00 boot page
AA55 external bridge
place field
result field
fold field
```

The implementation applies bytes through `omi_bb_apply()`, counting diagonal classes and updating place/result fields. It also detects bridge constants including `0x1E`, `0x78`, `0x7F`, `0x7C00`, and `0xAA55`.

Canonical reading:

```
The bitboard is a witness surface.
It summarizes envelope structure.
```

It does not replace the envelope.
It does not accept state.

3. Sector Layer

A sector is exactly eight OMI envelopes:

```
OMI_SECTOR_SIZE = 512 bytes  
OMI_CELLS_PER_SECTOR = 8
```

Each sector cell is one 64-byte envelope. The sector iterator parses and validates each cell, then stores a bitboard for valid cells.

Canonical reading:

```
512-byte sector  
  = 8 × 64-byte OMI envelopes  
  
sector iterator  
  = bounded envelope scanner  
  
valid cell  
  = envelope passed local validation  
  
valid cell  
  ≠ accepted state
```

The sector layer is a batch boundary, not an authority boundary.

4. Stream Layer

The stream parser is a byte-level state machine:

```
WAITING  
HEADER  
PAYLOAD  
COMPLETE  
INVALID
```

The parser waits for the canonical preheader, then fills a 64-byte envelope buffer. Once full, it validates the envelope and optionally dispatches it.

The implementation's canonical preheader is again:

```
FF 00 1C 1D 1E 1F 20 FF
```

The stream parser outputs a `StreamEvent` containing:

```
bitboard
envelope
valid flag
cell index
opcode
dispatch result
response buffer
```

Canonical reading:

```
The stream parser recognizes frames.
It may validate envelope shape.
It may dispatch when configured.
It does not itself define canon.
```

The stream layer is the byte-framing surface.

5. Orbit Layer: GL(16,2) Runtime Dynamics

The orbit implementation defines a four-layer algebra:

```
Layer 1: GL(16,2) linear dynamics
Layer 2: equivariant observables
Layer 3: BQF quadratic form
Layer 4: 5040 orbit atlas
```

The core delta in `omi_orbit.c` is:

```
 $\Delta(x, c) = A[x] \text{ XOR } B[c]$ 
```

where `A` is multiplication by a primitive element in `GF(216)` using the polynomial:

$$x^{16} + x^5 + x^3 + x^2 + 1$$

The runtime precomputes **A** and **B**, traces orbits, and detects cycles over 16-bit state.

Important distinction:

```
omi_orbit delta
  = GL(16,2) / GF(2^16) linear operator

omi_sense / omi_pg delta
  = rotation-XOR transduction operator
```

These are related runtime deltas, but they are not the same implementation.

The orbit layer also defines observables:

```
Fano      = x mod 7
Tetra     = x mod 4
Phase     = x mod 2
BQF       = 60x2 + 16xy + 4y2
Slot5040  = fano × 720 + tetra × 240 + phase
```

The orbit header names this as a four-layer architecture and states the core theorem as equivariance under **GL(16,2)**.

Canonical reading:

```
Orbit gives bounded algebraic runtime coordinates.
Orbit attestations are witnesses.
They are not receipts by themselves.
```

6. Projective Geometry Layer: PG(4,2) Incidence Kernel

The PG layer models:

```
PG(4,2)
31 points
```

```
155 lines
128-bit incidence kernel
```

The incidence kernel is stored in:

```
relation[16..31]
```

The header defines the kernel as a 128-bit value computed from active gauge points and their line closure.

The implementation:

```
extracts active PG points from gauge[i] & 0x1F
computes lines {p, q, p XOR q}
XORs encoded line contributions
stores/loads kernel from relation tail
```

The PG layer also applies a delta safety filter:

```
F(x) = Δ(x, C) XOR x
C = 0x55AA
```

A subblock is unsafe if:

```
F(x) == 0
or
F(F(x)) == x
```

That rejects fixed points and 2-cycle collapses.

Canonical reading:

```
PG(4,2) supplies projective consistency checks.
The kernel is a cross-reference witness.
The kernel is not identity.
The kernel is not receipt.
```

7. Delta Family Distinction

The current runtime has at least two delta forms.

7.1 Orbit Delta

```
Δ_orbit(x,c) = A[x] XOR B[c]
```

This is a `GL(16,2)` linear map implemented through `GF(2^16)` multiplication.

7.2 Sense / PG Delta

```
Δ_sense(x,C) =  
    rotl16(x,1)  
    XOR rotl16(x,3)  
    XOR rotr16(x,2)  
    XOR C
```

This appears in `omi_sense_delta16()` and is mirrored internally by `omi_pg.c`.

Canonical rule:

Do not collapse the delta family into one unnamed operator.

Δ_orbit
 algebraic GL(16,2) step

Δ_sense
 bitwise rotation-XOR transduction

Δ_pg
 local mirror of Δ_sense for projective safety checks

This distinction should be preserved in canon docs.

8. Sense Layer: Light Garden Frame

The sense layer defines a raw transducer frame:

```
typedef struct {  
    uint8_t  preheader[8];  
    uint32_t cycle;  
    uint32_t source_id;  
    uint32_t sensor_mask;  
    uint32_t geometry_mask;
```



```

uint32_t orientation;
uint32_t sample_word;
uint32_t delta_word;
uint32_t omi_witness;
uint32_t imo_witness;
uint32_t flags;
} OMI_SenseFrame;

```

It also defines effect classes:

```

PURE
READ_ONLY
LOCAL_WRITE
REPO_WRITE
NETWORK
HARDWARE
CAPTURE
SECURITY

```

and sensor/geometry masks for light, sound, touch, gesture, motion, orientation, proximity, camera, environment, network, DOM, hardware, ring, personal, garden, radial, tetrahedral, 5-cell, 11-cell, and orbital geometry.

The sense frame maps into the OMI envelope as:

```

gauge      = preheader
orientation = cycle + source_id
recovery    = orientation + sensor_mask
target      = geometry_mask + sample_word
relation    = delta_word + omi_witness + imo_witness + flags

```

The validator checks the preheader, sensor mask, geometry mask, and orientation bounds. It explicitly validates candidate frame shape; it does not accept state.

Canonical reading:

```

Sense frames transduce raw signals into envelope candidates.
They are candidate carriers.
They are not accepted state.

```

9. Omicron Layer: Slide and Projection

The omicron layer resolves envelope values into layered coordinates:

```
physical  
route  
declaration  
witness  
phase
```

The implementation supports layers 1 through 12. Layers 1–7 are derived from bitboard/factorial-style resolutions, while layers 8–12 derive physical, route, declaration, witness, and phase values from gauge/orientation fields.

The header names:

```
OMICRON_LAYER_PHYSICAL = 8  
OMICRON_LAYER_ROUTE    = 9  
OMICRON_LAYER_DECL     = 10  
OMICRON_LAYER_WITNESS  = 11  
OMICRON_LAYER_PHASE    = 12
```

It also defines a four-phase slide:

```
PLACE  
REGISTER_INJECT  
KERNEL_READ  
RECORD_CLOSE
```

with codes:

```
0x00  
0x1C  
0x1E  
0x20
```

Canonical reading:

Omicron resolves layered projection values.
Omicron slide coordinates phase traversal.
Projection is not authority.

10. Transport Layer

The generic transport ABI is:

```
struct OMI_Transport {  
    int (*send)(...);  
    int (*recv)(...);  
    int (*available)(...);  
    int (*flush)(...);  
    void* ctx;  
};
```

It provides send/receive helpers for full `OMI_512_Envelope` values.

The simulated transport is a linked in-memory buffer transport. It implements `send`, `recv`, `available`, and `flush` using a peer buffer.

The LoRa transport exposes configuration for SPI pins, frequency, spreading factor, bandwidth, coding rate, and TX power. Its implementation appends/checks CRC around a 64-byte envelope, but current send behavior is stub-like and returns failure after preparing the wire buffer.

Canonical reading:

Transport moves envelopes.
Transport does not define envelope truth.
Transport does not accept state.
Transport may fail independently of canonical validity.

11. Receipt Layer

The receipt layer computes a receipt from:

```
omicron layer values
gauge action
path bytes from gauge + orientation
```

It can verify a receipt by recomputing those values and comparing them.

The receipt header defines:

```
OMI_RECEIPT_PATH_MAX = 16
OMI_RECEIPT_LAYERS = 5
```

and the receipt structure:

```
layer_values[5]
gauge_action
path[16]
path_len
valid
```

The receipt layer also has `omi_receipt_verify_meta64()`, which checks the canonical preheader and FS/GS/RS/US positions.

Canonical reading:

```
Receipt is the first layer that explicitly records accepted projection shape.
Receipt verification is separate from parsing, streaming, transport, and
projection.
```

Important caution:

```
A computed receipt is not the same as social or protocol consensus.
It is a local receipt object unless promoted by the surrounding validation/
recording regime.
```

12. OMIOM Prefix Resolver

The OMIOM resolver implements prefix entries:

```
address
prefix_len
next_hop
layer
valid
```

It supports add, remove, resolve, cascade, and count. The direct resolver performs longest-prefix matching against the target-derived 64-bit address. The cascade fallback uses omicron route/declaration/witness/phase layers.

Canonical reading:

```
OMIOM is route selection by prefix and cascade.
It resembles CIDR-style resolution.
It is not validation.
It is not receipt.
```

This aligns with the newer `omi-port` direction:

```
URI-CIDR / prefix scope
  declares routing scope

OMIOM prefix resolver
  chooses next hop over envelope target/layers

Validation/receipt
  remain separate
```

13. Parser Layer

The parser shown here is a minimal Lisp-style pair parser:

```
symbol
(car . cdr)
```

It consumes symbols and dotted pairs, producing atoms and pairs.

Canonical reading:

```
Parser declares structure.  
Parser does not evaluate.  
Parser does not validate.  
Parser does not receipt.
```

This aligns with `omi-lisp`:

```
parse  
→ candidate  
→ adapter node  
→ boundary shape
```

No parser should be treated as authority.

14. Canonical Runtime Ladder

The attached runtime files imply this ladder:

```
raw bytes  
→ stream parser  
→ OMI_512_Envelope  
→ local envelope validation  
→ bitboard witness  
→ sector/event grouping  
→ orbit / PG / sense / omicron projections  
→ transport movement  
→ receipt computation/verification
```

Authority separation:

```
stream recognition  
  frame found  
  
envelope validation  
  shape locally valid  
  
bitboard witness  
  compressed evidence  
  
PG kernel  
  projective consistency
```

sense frame
transducer candidate

transport
carrier movement

receipt
recorded projection proof

canon
names invariant boundaries

15. Normative vs Candidate Status

Normative / Canon-worthy

512-bit envelope shape
canonical preheader
FS/GS/RS/US control ladder
64-byte envelope / 512-byte sector relationship
bitboard witness vocabulary
orbit layer distinction
PG(4,2) incidence kernel role
sense-frame-to-envelope mapping
transport-as-carrier boundary
receipt separate from parsing/transport/projection

Candidate / Runtime-specific

specific GF(2¹⁶) primitive polynomial
specific mod-7/mod-4/mod-2 quotient observer choices
specific BQF formula
specific 5040 slot formula
specific PG point seed constant
specific route resolver fallback order
specific LoRa stub behavior

Not Canon Authority

transport success
parser success

```
stream event
bitboard compression
projection display
computed local receipt without surrounding acceptance regime
```

16. Open Issues Before Canon Freeze

1. Delta naming must be disambiguated.

```
omi_orbit_delta
omi_sense_delta
omi_pg_delta
```

should not be collapsed into one `delta16` doctrine.

1. Receipt authority must be scoped.

`omi_receipt_compute()` creates a receipt object, but canon should distinguish:

```
computed receipt
verified receipt
accepted receipt
repository receipt
network receipt
```

1. LoRa transport is not complete authority.

The LoRa implementation prepares CRC-wrapped wire data but send remains stub-like. It should be documented as a carrier stub until hardware behavior is complete.

1. Parser is too minimal to be canonical OMI-Lisp.

The provided parser is useful as a dotted-pair parser, but `omi-lisp` owns the fuller candidate ladder.

1. OMIOM and omi-port should be aligned.

OMIOM already has prefix matching. `omi-port` has URI-CIDR source/target scopes. Canon should define how these relate without importing one repo into the other.

17. One-Page Canon Summary

OMI runtime begins with a 64-byte envelope.

The envelope begins with:

FF 00 1C 1D 1E 1F 20 FF

The control ladder is:

NUL FS GS RS US SP

A 512-byte sector holds eight envelopes.

A stream parser recognizes envelopes from byte flow.

A bitboard compresses witness features.

Orbit gives $GL(16,2)$ runtime coordinates.

$PG(4,2)$ gives projective incidence consistency.

Sense frames transduce physical signals into envelope candidates.

Omicron resolves projection layers and slide phases.

Transport moves envelopes.

Receipt records projection evidence.

None of these alone creates canonical acceptance.

18. Final Doctrine

The OMI runtime is a bounded envelope calculus.

It stages memory topology before language.

It carries sign-value data through fixed-size frames.

It projects orbit, geometry, sense, and route witnesses.

It may transport envelopes across carriers.

It may compute receipts.

But only the validation and receipt authority boundary may accept state.

Short form:

Envelope carries.
Stream frames.
Sector batches.
Orbit coordinates.
PG constrains.
Sense transduces.
Omicron projects.
Transport moves.
Receipt records.
Canon names the invariant.